

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Д. О. Шевяков
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Организация получения и вывода данных
с удаленного оборудования

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание	3
3 Ход выполнения задания.....	4
4 Вывод.....	6
ПРИЛОЖЕНИЕ А	7

1 Цель работы

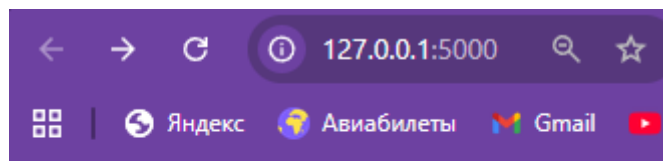
Целью лабораторной работы является получение практических навыков организации получения данных с удалённого оборудования и вывода этих данных в веб-интерфейсе. В рамках варианта «Умный дом» в качестве оборудования используется программный эмулятор: состояние вещей периодически обновляется на сервере, а клиент запрашивает данные мониторинга по отдельным маршрутам.

2 Задание

В соответствии с методическими указаниями для лабораторной работы №3 требуется реализовать эмуляторы вещей для всех классов, созданных в предыдущей работе, разработать один или несколько интерфейсов для вывода данных от эмуляторов. В рамках данной работы в интерфейс передаются только данные мониторинга; передача управляющих команд рассматривается в следующей лабораторной работе. Для каждой вещи необходимо реализовать собственный запрос к серверу и собственную функцию подключения на стороне сервера.

3 Ход выполнения задания

В ходе выполнения лабораторной работы на сервере Node.js с использованием Express организована раздача статических файлов из каталога `public` (страница `index.html` и клиентские сценарии). Для каждой вещи (`TemperatureSensor`, `LightController`, `SmartLock`) создан отдельный маршрут `GET /connect/...`, возвращающий JSON с полями мониторинга после вызова метода `connect()` цифрового двойника. В базовом классе `Thing` реализован общий сценарий `connect()`: вызов `emulate()` и формирование полезной нагрузки через `buildPayload()` в наследниках. На рисунке 1 изображена структура клиентской части и результат мониторинга в браузере (страница с обновляющимися значениями температуры, света и замка).



Мониторинг (ЛР3)

Температура

42.1 °C

Свет

Состояние: ВЫКЛ

Яркость: 0%

Замок

ОТКРЫТ

Рисунок 1 – Главная страница мониторинга с видимыми значениями

На стороне клиента в файлах `script.js` и `index.js` реализованы функции периодического опроса сервера с интервалом одна секунда (`setInterval`), для каждой вещи выполняется отдельный запрос `fetch` к соответствующему маршруту `/connect/temperature`, `/connect/light`, `/connect/lock`, полученный JSON используется для обновления элементов страницы. На рисунке 2

изображены сетевые запросы к маршрутам подключения (вкладка «Сеть» / «Network» инструментов разработчика).

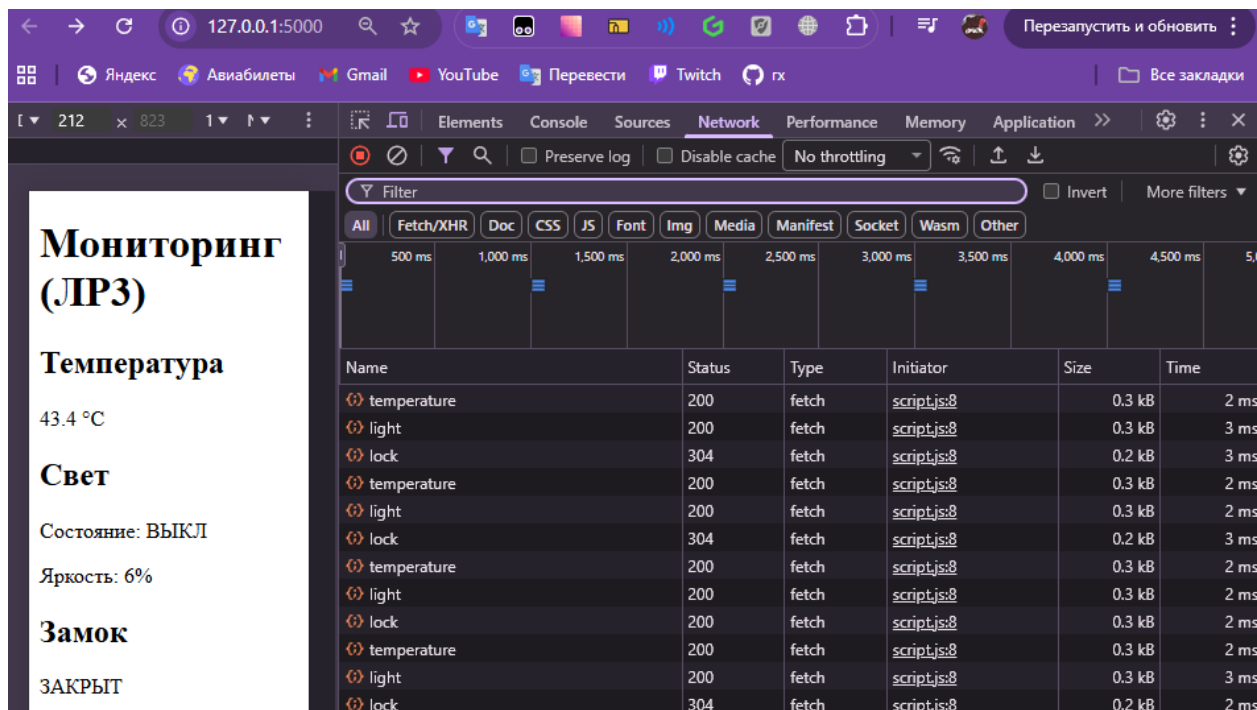


Рисунок 2 - Скриншот вкладки Network: видны запросы к `'/connect/temperature'`, `'/connect/light'`, `'/connect/lock'` со статусом 200.

При каждом запросе на стороне сервера вызываются методы вещей, что позволяет зафиксировать работу эмуляции и выдачи данных в журнале консоли процесса Node.js. На рисунке 3 изображён фрагмент вывода консоли сервера при работе страницы мониторинга.

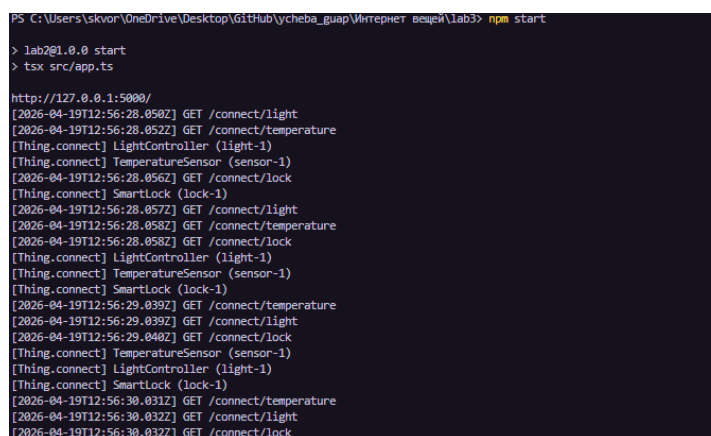


Рисунок 3 - Скриншот терминала с логами сервера во время обновления страницы

4 Вывод

В результате выполнения лабораторной работы была реализована схема получения данных мониторинга с эмулируемого оборудования через отдельные HTTP-маршруты для каждой вещи, организован вывод данных во веб-интерфейсе с периодическим опросом сервера. Использование общего метода `connect()` в абстрактном классе `Thing` и индивидуальных реализаций `emulate()` и `buildPayload()` в наследниках соответствует выбранному варианту проектирования и требованиям методических указаний к лабораторной работе №3. Реализация может быть использована как основа для следующей работы по передаче управляющих команд.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
// app.ts
import express from "express";
import path from "node:path"
import { TemperatureSensor, LightController, SmartLock } from "./things";

const app = express();
const PORT = 5000;

const temp = new TemperatureSensor("sensor-1", "Температурный датчик", 22.5);
const light = new LightController("light-1", "Свет в гостиной");
const lock = new SmartLock("lock-1", "Замок входной двери");

app.use(express.static(path.join(process.cwd(), "public")));

app.get("/connect/temperature", (_req, res) => res.json(temp.connect()))
app.get("/connect/light", (_req, res) => res.json(light.connect()))
app.get("/connect/lock", (_req, res) => res.json(lock.connect()))

app.listen(PORT, () => {
  console.log(`http://127.0.0.1:${PORT}/`)
});

// things.ts
export abstract class Thing {
  constructor(
    public id: string,
```



```
    public name: string,  
    public isOnline: boolean = true  
  ) {}
```

```
protected abstract emulate(): void  
protected abstract buildPayload(): Record<string, unknown>;
```

```
connect() {  
    this.emulate();  
    return this.buildPayload();  
}
```

```
abstract getStatus(): string;  
}
```

```
export interface IParent {  
    render(statuses: string[]): string;  
}
```

```
export interface IChild {  
    render(statuses: string[]): string;  
}
```

```
export class MainControlUnit {  
    constructor(  
        public readonly things: Thing[],  
        public readonly datastore: DeviceDataStore  
    ) {}
```

```
    registerThing(thing: Thing): void {
```

```

        console.log(`[MainControlUnit.registerThing] ${thing.id}`);
        this.things.push(thing);
    }
    collectMonitoringData(): void {
        console.log(`[MainControlUnit.collectMonitoringData]` );
        for (const thing of this.things) {
            const line = `[${new Date().toISOString()}] ${thing.getStatus()}`
            this.dataStore.saveRecord(thing, line)
        }
    }
    getAllStatuses(): string[] {
        console.log(`[MainControlUnit.getAllStatuses]`);
        return this.things.map((t) => t.getStatus());
    }
}

```

```

export class DeviceDataStore {
    private readonly dataByThingId = new Map<string, string[]>();

    saveRecord(thing: Thing, payload: string): void {
        console.log(`[DeviceDataStore.saveRecord] ${thing.id}` );
        const list = this.dataByThingId.get(thing.id) ?? [];
        list.push(payload);
        this.dataByThingId.set(thing.id, list);
    }

    getRecords(thingId: string): string[] {
        console.log(`[DeviceDataStore.getRecords] ${thingId}` );
        return this.dataByThingId.get(thingId) ?? [];
    }
}

```

```
}
```

```
export class TemperatureSensor extends Thing {  
  private currentTemperatureC: number;  
  
  constructor(id: string, name: string, initialTemperatureC: number) {  
    super(id, name, true);  
    this.currentTemperatureC = initialTemperatureC;  
  }  
  
  setTemperature(valueC: number): void {  
    console.log(`[TemperatureSensor.setTemperature] ${this.id}`);  
    this.currentTemperatureC = valueC;  
  }  
  
  getTemperature(): number {  
    return this.currentTemperatureC;  
  }  
  
  protected emulate(): void {  
    const delta = (Math.random() - 0.5) * 1.2;  
    this.currentTemperatureC = Math.round((this.currentTemperatureC + delta) *  
10) / 10;  
  }  
  
  protected buildPayload(): Record<string, unknown> {  
    return {  
      id: this.id,  
      name: this.name,
```

```

        value: this.currentTemperatureC,
        unit: "°C"
    };
}

```

```

getStatus(): string {
    console.log(`[TemperatureSensor.getStatus] ${this.id}`);
    return `${this.name}: ${this.currentTemperatureC.toFixed(1)} °C`
}
}

```

```

export class LightController extends Thing {
    private brightnessPercent: number = 0;
    private isEnabled: boolean = false;

    setLight(isEnabled: boolean, brightnessPercent: number): void {
        console.log(`[LightController.setLight] ${this.id}`);
        this.isEnabled = isEnabled;
        this.brightnessPercent = Math.max(0, Math.min(100, brightnessPercent))
    }
}

```

```

protected emulate(): void {
    const delta = Math.floor((Math.random() - 0.5) * 12);
    this.brightnessPercent = Math.max(0, Math.min(100, this.brightnessPercent +
delta));
}

```

```

protected buildPayload(): Record<string, unknown> {
    return {

```

```

        id: this.id,
        name: this.name,
        enabled: this.isEnabled,
        brightnessPercent: this.brightnessPercent
    };
}

getStatus(): string {
    console.log(`[LightController.getStatus] ${this.id}`);
    return `${this.name}: ${this.isEnabled ? "ВКЛ" : "ВЫКЛ"},
яркость=${this.brightnessPercent}%`;
}
}

export class SmartLock extends Thing{
    private isLocked: boolean = true;

    setLocked(value: boolean): void {
        console.log(`[SmartLock.setLocked] ${this.id}`);
        this.isLocked = value;
    }

    protected emulate(): void {
        if (Math.random() < 0.08) this.isLocked = !this.isLocked;
    }

    protected buildPayload(): Record<string, unknown> {
        return {
            id: this.id,
            name: this.name,

```

```

        locked: this.isLocked
    };
}

getStatus(): string {
    console.log(`[SmartLock.getStatus] ${this.id}`);
    return `${this.name}: ${this.isLocked ? "ЗАКРЫТ" : "ОТКРЫТ"}`;
}
}

export class ParentInterface implements IParent {
    render(statuses: string[]): string {
        console.log("[ParentInterface.render]");
        return ["Интерфейс родителя (полный доступ)", ...statuses].join("\n");
    }
}

export class ChildInterface implements IChild {
    render(statuses: string[]): string {
        console.log("[ChildInterface.render]");
        return ["Интерфейс ребенка (ограничен)", ...statuses].join("\n");
    }
}

export function runDemo(): {
    parentText: string;
    childText: string;
    sensorRecords: string[];
} {
    const store = new DeviceDataStore();

```

```

const things: Thing[] = [];
const mcu = new MainControlUnit(things, store);
  const temp = new TemperatureSensor("sensor-1", "Температурный датчик",
22.5);
const light = new LightController("light-1", "Свет в гостиной");
const lock = new SmartLock("lock-1", "Замок входной двери");
light.setLight(true, 65);
lock.setLocked(true);
mcu.registerThing(temp);
mcu.registerThing(light);
mcu.registerThing(lock);
mcu.collectMonitoringData();
const statuses = mcu.getAllStatuses();
const parentText = new ParentInterface().render(statuses);
const childText = new ChildInterface().render(statuses);
return {
  parentText,
  childText,
  sensorRecords: store.getRecords("sensor-1")
};
}

```

//script.js

```

const CONNECT = {
  temperature: "/connect/temperature",
  light: "/connect/light",
  lock: "/connect/lock",
};

```

```

async function fetchJson(url) {

```

```

const response = await fetch(url);
if (!response.ok) {
  throw new Error(`HTTP ${response.status} для ${url}`);
}
return response.json();
}

//index.js
function $(id) {
  return document.getElementById(id);
}

async function updateTemperature() {
  const data = await fetchJson(CONNECT.temperature)
  document.getElementById("tempValue").textContent = `${data.value}
  ${data.unit}`
}

async function updateLight() {
  const data = await fetchJson(CONNECT.light);
  $("lightEnabled").textContent = data.enabled ? "ВКЛ" : "ВЫКЛ";
  $("lightBrightness").textContent = String(data.brightnessPercent);
}

async function updateLock() {
  const data = await fetchJson(CONNECT.lock);
  $("lockState").textContent = data.locked ? "ЗАКРЫТ" : "ОТКРЫТ";
}

async function tick() {
  await Promise.all([updateTemperature(), updateLight(), updateLock()]);

```



```
}
```

```
tick();
```

```
setInterval(tick, 1000);
```

```
//index.html
```

```
<!DOCTYPE html>
```

```
<html lang="ru">
```

```
<head>
```

```
  <meta charset="UTF-8" />
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
  <title>IoT Дом — ЛР3</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Мониторинг (ЛР3)</h1>
```

```
  <section>
```

```
    <h2>Температура</h2>
```

```
    <p><span id="tempValue">—</span></p>
```

```
  </section>
```

```
  <section>
```

```
    <h2>Свет</h2>
```

```
    <p>Состояние: <span id="lightEnabled">—</span></p>
```

```
    <p>Яркость: <span id="lightBrightness">—</span>%</p>
```

```
  </section>
```

```
  <section>
```

```
    <h2>Замок</h2>
```

```
    <p><span id="lockState">—</span></p>
```

</section>

<script src="/script.js"></script>

<script src="/index.js"></script>

</body>

</html>